COSC 6385 - Homework 1: Out-Of-Order (O3) Simulation

Nicholas Anderson

1 Problem Statement

For this assignment we were given sample python code which simulates out of order exeuctions using the tomasulo algorithm. This algorithm includes reorder buffer (ROB), register alias table (RAT), a load-store-queue, reservation stations, functional units, and pipelining. All of these features come together to facilitate out-of-order execution. We were tasked with modifying this code to allow for multiple instructions to be executed during one clock cycle. Additionally I implemented allowance for commiting multiple instructions per clock cycle, as that is something that we talked about during the lecture in which we went over a practice problem closely related to this homework.

Additionally the input and output described in the assignment pdf is different from what is currently implemented. Specifically the existing input cannot take in register values, number of reservation stations (RS), cycles in EX, cycles in MEM, number of functional units (FUs), ROB entries, and issue width from the input text file. The output does not output the final register and non-zero memory values. Therefore I added this functionality into the existing code.

2 Expanding Issue Width

For this assignment we expand issue width to be able to issue and commit up to 4 instructions in one cycle. Expanding the amount of commands which can be issued in a cycle can be done directly through main by modifying the issue stage (lines 90-105 in main.py). This is done by adding a for loop of range issue_width, and issuing an instruction each iteration as long as the issuing is valid (i.e. there is an open RS for the instruction) and there are still instructions to be issued.

Expanding issue with for commits as well is done through the `commit` function in commit.py. Again this is done simply by adding a for loop of range issue_width, and committing an instruction each iteration as long is the head pointer of the ROB is pointing at a instruction that is ready to commit. Since instructions must commit in order, as soon as the instruction which the head pointer points to (implemented as `ROB[0]`) is not ready to commit, we break from the fop loop so no further operations are committed and the current operation is not removed from the ROB.

3 Input Parser

An input parser is used to be read from the input file the structure of our o3-simulator, that being the number of RS' for each instruction type (integer adder, FP adder/multiplier, and load/stores), The number of cycles each FU takes to execute and store in memory (memory is only impacted by load/store instructions, all other instructions take 0 cycles in the MEM pipeline stage), number of ROB entries, the issue width, and initial memory and register values. This is implemented in input_parser.py though standard parser methods (i.e. reading the input line by line and using regular expressions to parse values). Parsed values are placed into the `SimulationInput`. A `UnitConfig` class is also used, which `SimulationInput` derives values from to save

the full information of the program. A `SimulationInput` instance is returned to main through `read_input(filename)` (function in input_parser.py) containing all the information required to run the simulation with the parameters given in the input file.

4 Test Cases/Results

All test cases and results can be found in "test_cases" and "results" directories located in 'hw1/o3-simulator/'.

4.1 test_case.txt

The first test case we look at is the one given to us in test_case.txt, which has the following parameters for the input (issue width is not given as we change it to show how it impact runs):

Int. Adder: 4 RS, 1 Cycle in EX, 1 FU
FP Adder: 3 RS, 4 Cycles in EX, 1 FU
FP Mult.: 2 RS, 15 Cycles in EX, 1 FU
Load/Store: 5 RS, 1 Cycle in EX, 5 Cycles in Mem, 1 FU
ROB Entries: 64
Starting Register Values: R1=12, R2=32, F20=3.0
Starting Memory Values: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

The results of changing the issue width between 1,2,3, and 4 can be seen in figures 1, 2, 3, and 4. The main findings are that increasing issue width cannot increase the cycle of the final commit, however it can increase the cycle of intermediate commits. This is due to the way pipelining is implemented. Since EXE comes before MEM, instructions which do not require MEM will perform the WB step first since they are put into the results buffer during the `exe(...)` function call. Since the first instrution is a load operation, if non load/store instructions finish the EXE step on the same cycle that it finishes its MEM step, the non load/store instructions are placed into the results buffer first. This occurs when the issue width is 1, 3, and 4. Addi instructions finish the EXE stage on the same cycle which instruction 1 finishes its MEM stage. Since the Addi instructions (specifically their results) are placed into the results buffer first, they perform the WB stage first. Since only one instruction can perform the WB stage at one time, this means the first instruction has to wait for a later cycle. We can see by looking at 2 that an issue width of 2 avoids this, since instruction 6 finishes before cycle 7 (when instruction 1 finishes its MEM stage), and instructions 10 and 11 are not issued quickly enough to finish on cycle 7, the first instruction can perform the WB stage in the cycle directly after it finishes its MEM step.

4.2 test_case2.txt

In this case we stress test all portions of the o3 simulator to make sure that everything is working correctly. In total there are 73 instructions issued, memory loads and stores, all FUs and instruction types are used. The hardware configuration is given like so:

Int. Adder: 2 RS, 1 Cycle in EX, 1 FU
FP Adder: 2 RS, 3 Cycles in EX, 1 FU
FP Mult.: 2 RS, 15 Cycles in EX, 1 FU
Load/Store: 3 RS, 1 Cycle in EX, 1 Cycles in Mem, 1 FU
ROB Entries: 28
Starting Register Values: R4=10, R5=3, R15=2, F1=2.0, F2=3.0, F3=4.0, F4=5.0, F5=6.0, F6=7.0, F7=8.0
Starting Memory Values: Mem[0]=10.0, Mem[4]=20.0, Mem[8]=30.0, Mem[12]=40.0

As the outputs are extremely long, I have included the output for an issue width of 1 and 4 in the Appendix sections 6.2 and 6.3 respectively. In this example again we can see that similarly to the previous test case,

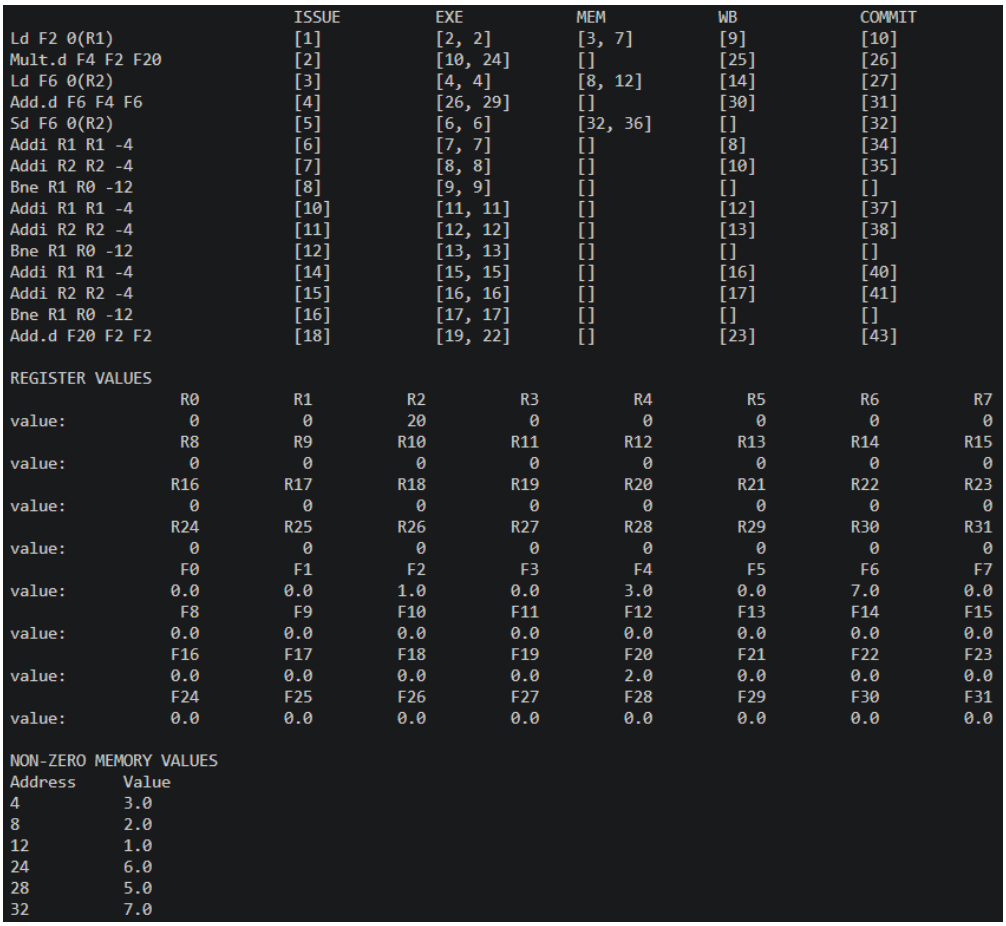
increasing the issue width allows for faster execution. With an issue width of 1 execution takes 130 cycles, whereas with an issue width of 4 execution takes only 105 cycles while resulting in the same final register and non-zero memory values.

5 AI Usage

I worked closely with codex 6.1 for this assignment. The link to the chat log is here: **Chat log link**. The work it performed included giving me an overview of the required changes, implementing those changes, creating test_case2.txt, and fixing a bug which allowed for out-of-order commits to occur.

6 Appendix

6.1 test_case.txt figues



	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[9]	[10]
Mult.d F4 F2 F20	[2]	[10, 24]	[]	[25]	[26]
Ld F6 0(R2)	[3]	[4, 4]	[8, 12]	[14]	[27]
Add.d F6 F4 F6	[4]	[26, 29]	[]	[30]	[31]
Sd F6 0(R2)	[5]	[6, 6]	[32, 36]	[]	[32]
Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[34]
Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[35]
Bne R1 R0 -12	[8]	[9, 9]	[]	[]	[]
Addi R1 R1 -4	[10]	[11, 11]	[]	[12]	[37]
Addi R2 R2 -4	[11]	[12, 12]	[]	[13]	[38]
Bne R1 R0 -12	[12]	[13, 13]	[]	[]	[]
Addi R1 R1 -4	[14]	[15, 15]	[]	[16]	[40]
Addi R2 R2 -4	[15]	[16, 16]	[]	[17]	[41]
Bne R1 R0 -12	[16]	[17, 17]	[]	[]	[]
Add.d F20 F2 F2	[18]	[19, 22]	[]	[23]	[43]

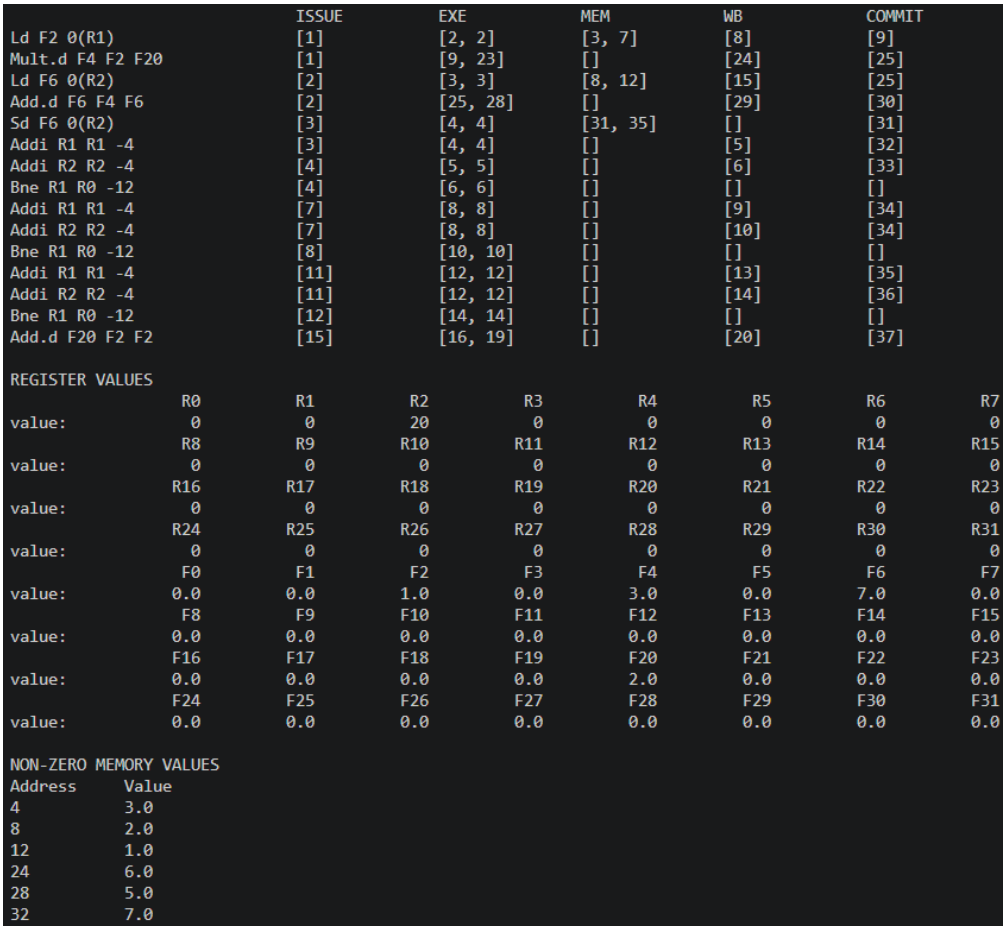
REGISTER VALUES

	R0	R1	R2	R3	R4	R5	R6	R7
value:	0	0	20	0	0	0	0	0
	R8	R9	R10	R11	R12	R13	R14	R15
value:	0	0	0	0	0	0	0	0
	R16	R17	R18	R19	R20	R21	R22	R23
value:	0	0	0	0	0	0	0	0
	R24	R25	R26	R27	R28	R29	R30	R31
value:	0	0	0	0	0	0	0	0
	F0	F1	F2	F3	F4	F5	F6	F7
value:	0.0	0.0	1.0	0.0	3.0	0.0	7.0	0.0
	F8	F9	F10	F11	F12	F13	F14	F15
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	F16	F17	F18	F19	F20	F21	F22	F23
value:	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0
	F24	F25	F26	F27	F28	F29	F30	F31
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

NON-ZERO MEMORY VALUES

Address	Value
4	3.0
8	2.0
12	1.0
24	6.0
28	5.0
32	7.0

Figure 1: Results from test_case.txt with an issue width of 1.



	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6 0(R2)	[2]	[3, 3]	[8, 12]	[15]	[25]
Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6 0(R2)	[3]	[4, 4]	[31, 35]	[]	[31]
Addi R1 R1 -4	[3]	[4, 4]	[]	[5]	[32]
Addi R2 R2 -4	[4]	[5, 5]	[]	[6]	[33]
Bne R1 R0 -12	[4]	[6, 6]	[]	[]	[]
Addi R1 R1 -4	[7]	[8, 8]	[]	[9]	[34]
Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[34]
Bne R1 R0 -12	[8]	[10, 10]	[]	[]	[]
Addi R1 R1 -4	[11]	[12, 12]	[]	[13]	[35]
Addi R2 R2 -4	[11]	[12, 12]	[]	[14]	[36]
Bne R1 R0 -12	[12]	[14, 14]	[]	[]	[]
Add.d F20 F2 F2	[15]	[16, 19]	[]	[20]	[37]

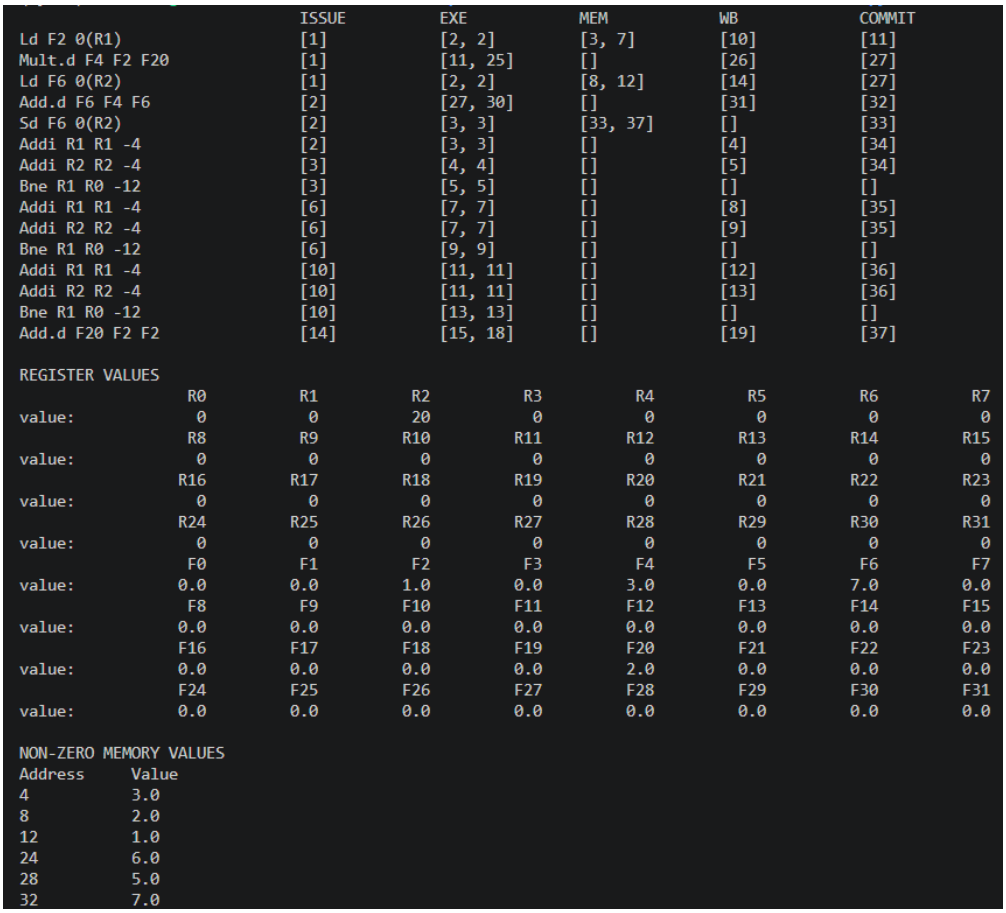
REGISTER VALUES

	R0	R1	R2	R3	R4	R5	R6	R7
value:	0	0	20	0	0	0	0	0
	R8	R9	R10	R11	R12	R13	R14	R15
value:	0	0	0	0	0	0	0	0
	R16	R17	R18	R19	R20	R21	R22	R23
value:	0	0	0	0	0	0	0	0
	R24	R25	R26	R27	R28	R29	R30	R31
value:	0	0	0	0	0	0	0	0
	F0	F1	F2	F3	F4	F5	F6	F7
value:	0.0	0.0	1.0	0.0	3.0	0.0	7.0	0.0
	F8	F9	F10	F11	F12	F13	F14	F15
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	F16	F17	F18	F19	F20	F21	F22	F23
value:	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0
	F24	F25	F26	F27	F28	F29	F30	F31
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

NON-ZERO MEMORY VALUES

Address	Value
4	3.0
8	2.0
12	1.0
24	6.0
28	5.0
32	7.0

Figure 2: Results from test_case.txt with an issue width of 2.



	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[10]	[11]
Mult.d F4 F2 F20	[1]	[11, 25]	[]	[26]	[27]
Ld F6 0(R2)	[1]	[2, 2]	[8, 12]	[14]	[27]
Add.d F6 F4 F6	[2]	[27, 30]	[]	[31]	[32]
Sd F6 0(R2)	[2]	[3, 3]	[33, 37]	[]	[33]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[34]
Addi R2 R2 -4	[3]	[4, 4]	[]	[5]	[34]
Bne R1 R0 -12	[3]	[5, 5]	[]	[]	[]
Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[35]
Addi R2 R2 -4	[6]	[7, 7]	[]	[9]	[35]
Bne R1 R0 -12	[6]	[9, 9]	[]	[]	[]
Addi R1 R1 -4	[10]	[11, 11]	[]	[12]	[36]
Addi R2 R2 -4	[10]	[11, 11]	[]	[13]	[36]
Bne R1 R0 -12	[10]	[13, 13]	[]	[]	[]
Add.d F20 F2 F2	[14]	[15, 18]	[]	[19]	[37]

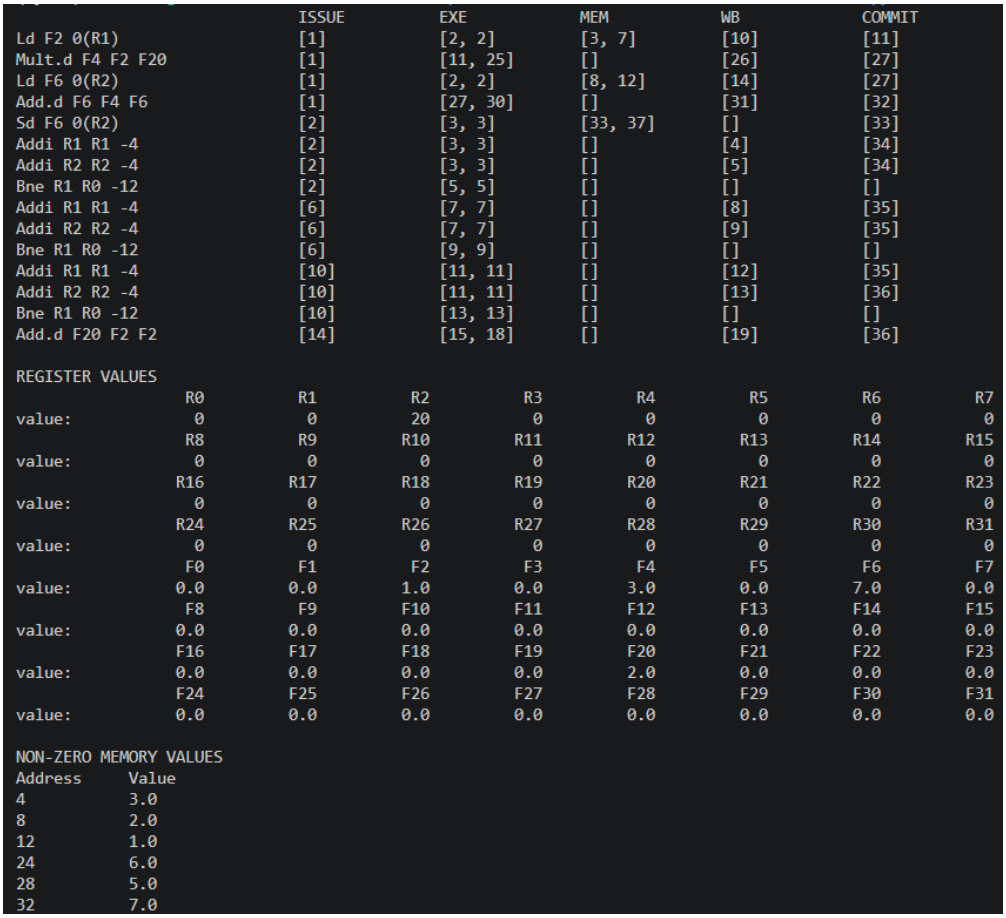
REGISTER VALUES

	R0	R1	R2	R3	R4	R5	R6	R7
value:	0	0	20	0	0	0	0	0
	R8	R9	R10	R11	R12	R13	R14	R15
value:	0	0	0	0	0	0	0	0
	R16	R17	R18	R19	R20	R21	R22	R23
value:	0	0	0	0	0	0	0	0
	R24	R25	R26	R27	R28	R29	R30	R31
value:	0	0	0	0	0	0	0	0
	F0	F1	F2	F3	F4	F5	F6	F7
value:	0.0	0.0	1.0	0.0	3.0	0.0	7.0	0.0
	F8	F9	F10	F11	F12	F13	F14	F15
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	F16	F17	F18	F19	F20	F21	F22	F23
value:	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0
	F24	F25	F26	F27	F28	F29	F30	F31
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

NON-ZERO MEMORY VALUES

Address	Value
4	3.0
8	2.0
12	1.0
24	6.0
28	5.0
32	7.0

Figure 3: Results from test_case.txt with an issue width of 3.



	ISSUE	EXE	MEM	WB	COMMIT
Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[10]	[11]
Mult.d F4 F2 F20	[1]	[11, 25]	[]	[26]	[27]
Ld F6 0(R2)	[1]	[2, 2]	[8, 12]	[14]	[27]
Add.d F6 F4 F6	[1]	[27, 30]	[]	[31]	[32]
Sd F6 0(R2)	[2]	[3, 3]	[33, 37]	[]	[33]
Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[34]
Addi R2 R2 -4	[2]	[3, 3]	[]	[5]	[34]
Bne R1 R0 -12	[2]	[5, 5]	[]	[]	[]
Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[35]
Addi R2 R2 -4	[6]	[7, 7]	[]	[9]	[35]
Bne R1 R0 -12	[6]	[9, 9]	[]	[]	[]
Addi R1 R1 -4	[10]	[11, 11]	[]	[12]	[35]
Addi R2 R2 -4	[10]	[11, 11]	[]	[13]	[36]
Bne R1 R0 -12	[10]	[13, 13]	[]	[]	[]
Add.d F20 F2 F2	[14]	[15, 18]	[]	[19]	[36]

REGISTER VALUES

	R0	R1	R2	R3	R4	R5	R6	R7
value:	0	0	20	0	0	0	0	0
	R8	R9	R10	R11	R12	R13	R14	R15
value:	0	0	0	0	0	0	0	0
	R16	R17	R18	R19	R20	R21	R22	R23
value:	0	0	0	0	0	0	0	0
	R24	R25	R26	R27	R28	R29	R30	R31
value:	0	0	0	0	0	0	0	0
	F0	F1	F2	F3	F4	F5	F6	F7
value:	0.0	0.0	1.0	0.0	3.0	0.0	7.0	0.0
	F8	F9	F10	F11	F12	F13	F14	F15
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	F16	F17	F18	F19	F20	F21	F22	F23
value:	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0
	F24	F25	F26	F27	F28	F29	F30	F31
value:	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

NON-ZERO MEMORY VALUES

Address	Value
4	3.0
8	2.0
12	1.0
24	6.0
28	5.0
32	7.0

Figure 4: Results from test_case.txt with an issue width of 4.

6.2 test_case2.txt output with issue width 1

	ISSUE	EXE	MEM	WB	COMMIT
Mult.d F8 F1 F2	[1]	[2, 16]	[]	[17]	[18]
Addi R10 R4 1	[2]	[3, 3]	[]	[4]	[19]
Add.d F9 F1 F2	[3]	[4, 6]	[]	[8]	[20]
Sd F9 64(R0)	[4]	[5, 5]	[22, 22]	[]	[22]
Sub R11 R4 R5	[5]	[6, 6]	[]	[7]	[23]
Sub.d F10 F4 F3	[6]	[7, 9]	[]	[11]	[24]
Sd F10 68(R0)	[7]	[8, 8]	[26, 26]	[]	[26]
Add R12 R4 R5	[8]	[9, 9]	[]	[10]	[27]
Add.d F11 F5 F6	[9]	[10, 12]	[]	[14]	[28]
Sd F11 72(R0)	[10]	[11, 11]	[30, 30]	[]	[30]
Addi R13 R4 2	[11]	[12, 12]	[]	[13]	[31]
Sub.d F12 F7 F1	[12]	[13, 15]	[]	[16]	[32]
Sd F12 76(R0)	[23]	[24, 24]	[34, 34]	[]	[34]
Add R14 R4 R5	[24]	[25, 25]	[]	[26]	[35]
Add.d F13 F2 F3	[25]	[26, 28]	[]	[29]	[36]
Sd F13 80(R0)	[27]	[28, 28]	[38, 38]	[]	[38]
Sub R16 R4 R5	[28]	[29, 29]	[]	[30]	[39]
Add.d F14 F4 F5	[29]	[30, 32]	[]	[33]	[40]
Sd F14 84(R0)	[31]	[32, 32]	[42, 42]	[]	[42]
Addi R17 R4 3	[32]	[33, 33]	[]	[34]	[43]
Sub.d F15 F6 F2	[33]	[34, 36]	[]	[37]	[44]
Sd F15 88(R0)	[35]	[36, 36]	[46, 46]	[]	[46]
Add R18 R4 R5	[36]	[37, 37]	[]	[38]	[47]
Add.d F16 F1 F7	[37]	[38, 40]	[]	[41]	[48]
Sd F16 92(R0)	[39]	[40, 40]	[50, 50]	[]	[50]
Sub R19 R4 R5	[40]	[41, 41]	[]	[42]	[51]
Ld F24 0(R0)	[43]	[44, 44]	[45, 45]	[46]	[52]
Ld F25 4(R0)	[45]	[46, 46]	[47, 47]	[48]	[53]
Ld F26 8(R0)	[47]	[48, 48]	[49, 49]	[50]	[54]
Ld F27 12(R0)	[48]	[49, 49]	[51, 51]	[53]	[55]
Add.d F17 F24 F25	[49]	[50, 51]	[]	[52]	[56]
Sub.d F18 F24 F26	[50]	[51, 53]	[]	[54]	[57]
Add.d F19 F24 F27	[51]	[54, 56]	[]	[58]	[59]
Mult.d F20 F17 F2	[52]	[53, 66]	[]	[67]	[68]
Mult.d F21 F17 F3	[53]	[55, 69]	[]	[71]	[72]
Mult.d F22 F17 F4	[54]	[55, 68]	[]	[69]	[73]
Addi R20 R4 1	[55]	[56, 56]	[]	[57]	[74]
Add R21 R20 R5	[56]	[58, 58]	[]	[59]	[75]
Sub R22 R20 R5	[57]	[60, 60]	[]	[62]	[76]
Addi R23 R20 2	[59]	[60, 60]	[]	[61]	[77]
Sd F20 128(R0)	[60]	[61, 61]	[79, 79]	[]	[79]
Ld F23 128(R0)	[61]	[62, 62]	[68]	[70]	[80]
Sd F18 132(R0)	[62]	[63, 63]	[82, 82]	[]	[82]
Sd F19 136(R0)	[69]	[70, 70]	[84, 84]	[]	[84]
Addi R1 R0 160	[70]	[71, 71]	[]	[72]	[85]
Sd F9 0(R1)	[80]	[81, 81]	[87, 87]	[]	[87]
Sd F9 32(R1)	[83]	[84, 84]	[89, 89]	[]	[89]
Addi R2 R0 164	[84]	[85, 85]	[]	[86]	[90]
Sd F10 0(R2)	[85]	[87, 87]	[92, 92]	[]	[92]
Sd F10 32(R2)	[88]	[89, 89]	[94, 94]	[]	[94]
Addi R3 R0 168	[89]	[90, 90]	[]	[91]	[95]
Sd F11 0(R3)	[90]	[92, 92]	[97, 97]	[]	[97]
Sd F11 32(R3)	[93]	[94, 94]	[99, 99]	[]	[99]
Addi R4 R0 172	[94]	[95, 95]	[]	[96]	[100]
Sd F12 0(R4)	[95]	[97, 97]	[102, 102]	[]	[102]
Sd F12 32(R4)	[98]	[99, 99]	[104, 104]	[]	[104]
Addi R5 R0 176	[99]	[100, 100]	[]	[101]	[105]
Sd F13 0(R5)	[100]	[102, 102]	[107, 107]	[]	[107]
Sd F13 32(R5)	[103]	[104, 104]	[109, 109]	[]	[109]
Addi R6 R0 180	[104]	[105, 105]	[]	[106]	[110]
Sd F14 0(R6)	[105]	[107, 107]	[112, 112]	[]	[112]
Sd F14 32(R6)	[108]	[109, 109]	[114, 114]	[]	[114]
Addi R7 R0 184	[109]	[110, 110]	[]	[111]	[115]
Sd F15 0(R7)	[110]	[112, 112]	[117, 117]	[]	[117]
Sd F15 32(R7)	[113]	[114, 114]	[119, 119]	[]	[119]

Addi R8 R0 188	[114]	[115, 115]	[]	[116]	[120]
Sd F16 0(R8)	[115]	[117, 117]	[122, 122]	[]	[122]
Sd F16 32(R8)	[118]	[119, 119]	[124, 124]	[]	[124]
Addi R15 R15 -1	[119]	[120, 120]	[]	[121]	[125]
Bne R15 R0 -8	[120]	[122, 122]	[]	[]	[126]
Addi R15 R15 -1	[123]	[124, 124]	[]	[125]	[127]
Bne R15 R0 -8	[124]	[126, 126]	[]	[]	[128]
Addi R30 R0 123	[127]	[128, 128]	[]	[129]	[130]

REGISTER VALUES

	R0	R1	R2	R3	R4	R5	R6	R7
value:	0	160	164	168	172	176	180	184
	R8	R9	R10	R11	R12	R13	R14	R15
value:	188	0	11	7	13	12	13	0
	R16	R17	R18	R19	R20	R21	R22	R23
value:	7	13	13	7	11	14	8	13
	R24	R25	R26	R27	R28	R29	R30	R31
value:	0	0	0	0	0	0	123	0
	F0	F1	F2	F3	F4	F5	F6	F7
value:	0.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0
	F8	F9	F10	F11	F12	F13	F14	F15
value:	6.0	5.0	1.0	13.0	6.0	7.0	11.0	4.0
	F16	F17	F18	F19	F20	F21	F22	F23
value:	10.0	30.0	-20.0	50.0	90.0	120.0	150.0	90.0
	F24	F25	F26	F27	F28	F29	F30	F31
value:	10.0	20.0	30.0	40.0	0.0	0.0	0.0	0.0

NON-ZERO MEMORY VALUES

Address	Value
0	10.0
4	20.0
8	30.0
12	40.0
64	5.0
68	1.0
72	13.0
76	6.0
80	7.0
84	11.0
88	4.0
92	10.0
128	90.0
132	-20.0
136	50.0
160	5.0
164	1.0
168	13.0
172	6.0
176	7.0
180	11.0
184	4.0
188	10.0
192	5.0
196	1.0
200	13.0
204	6.0
208	7.0
212	11.0
216	4.0
220	10.0

6.3 test_case2.txt output with issue width 4

	ISSUE	EXE	MEM	WB	COMMIT
Mult.d F8 F1 F2	[1]	[2, 16]	[]	[17]	[18]
Addi R10 R4 1	[1]	[2, 2]	[]	[3]	[18]

Add.d F9 F1 F2	[1]	[2, 4]	[]	[6]	[18]
Sd F9 64(R0)	[1]	[2, 2]	[20, 20]	[]	[20]
Sub R11 R4 R5	[2]	[3, 3]	[]	[4]	[20]
Sub.d F10 F4 F3	[2]	[3, 4]	[]	[8]	[20]
Sd F10 68(R0)	[2]	[3, 3]	[22, 22]	[]	[22]
Add R12 R4 R5	[2]	[4, 4]	[]	[7]	[22]
Add.d F11 F5 F6	[3]	[4, 5]	[]	[9]	[22]
Sd F11 72(R0)	[3]	[4, 4]	[24, 24]	[]	[24]
Addi R13 R4 2	[3]	[4, 4]	[]	[5]	[24]
Sub.d F12 F7 F1	[3]	[4, 6]	[]	[10]	[24]
Sd F12 76(R0)	[21]	[22, 22]	[26, 26]	[]	[26]
Add R14 R4 R5	[21]	[22, 22]	[]	[23]	[26]
Add.d F13 F2 F3	[21]	[22, 24]	[]	[26]	[27]
Sd F13 80(R0)	[23]	[24, 24]	[29, 29]	[]	[29]
Sub R16 R4 R5	[23]	[24, 24]	[]	[25]	[29]
Add.d F14 F4 F5	[23]	[24, 26]	[]	[28]	[29]
Sd F14 84(R0)	[25]	[26, 26]	[31, 31]	[]	[31]
Addi R17 R4 3	[25]	[26, 26]	[]	[27]	[31]
Sub.d F15 F6 F2	[25]	[26, 28]	[]	[30]	[31]
Sd F15 88(R0)	[27]	[28, 28]	[33, 33]	[]	[33]
Add R18 R4 R5	[27]	[28, 28]	[]	[29]	[33]
Add.d F16 F1 F7	[27]	[28, 30]	[]	[31]	[33]
Sd F16 92(R0)	[30]	[31, 31]	[35, 35]	[]	[35]
Sub R19 R4 R5	[30]	[31, 31]	[]	[32]	[35]
Ld F24 0(R0)	[32]	[33, 33]	[34, 34]	[35]	[36]
Ld F25 4(R0)	[34]	[35, 35]	[36, 36]	[37]	[38]
Ld F26 8(R0)	[34]	[35, 35]	[37, 37]	[38]	[39]
Ld F27 12(R0)	[36]	[37, 37]	[38, 38]	[39]	[40]
Add.d F17 F24 F25	[36]	[38, 39]	[]	[40]	[41]
Sub.d F18 F24 F26	[36]	[39, 40]	[]	[41]	[42]
Add.d F19 F24 F27	[39]	[40, 42]	[]	[43]	[44]
Mult.d F20 F17 F2	[39]	[41, 54]	[]	[55]	[56]
Mult.d F21 F17 F3	[39]	[43, 57]	[]	[59]	[60]
Mult.d F22 F17 F4	[42]	[43, 56]	[]	[57]	[60]
Addi R20 R4 1	[42]	[43, 43]	[]	[44]	[60]
Add R21 R20 R5	[42]	[47, 47]	[]	[49]	[60]
Sub R22 R20 R5	[43]	[45, 45]	[]	[46]	[61]
Addi R23 R20 2	[46]	[47, 47]	[]	[48]	[61]
Sd F20 128(R0)	[46]	[47, 47]	[63, 63]	[]	[63]
Ld F23 128(R0)	[46]	[47, 47]	[56]	[58]	[63]
Sd F18 132(R0)	[46]	[48, 48]	[65, 65]	[]	[65]
Sd F19 136(R0)	[57]	[58, 58]	[67, 67]	[]	[67]
Addi R1 R0 160	[57]	[58, 58]	[]	[60]	[67]
Sd F9 0(R1)	[64]	[65, 65]	[69, 69]	[]	[69]
Sd F9 32(R1)	[66]	[67, 67]	[71, 71]	[]	[71]
Addi R2 R0 164	[66]	[67, 67]	[]	[68]	[71]
Sd F10 0(R2)	[68]	[69, 69]	[73, 73]	[]	[73]
Sd F10 32(R2)	[70]	[71, 71]	[75, 75]	[]	[75]
Addi R3 R0 168	[70]	[71, 71]	[]	[72]	[75]
Sd F11 0(R3)	[72]	[73, 73]	[77, 77]	[]	[77]
Sd F11 32(R3)	[74]	[75, 75]	[79, 79]	[]	[79]
Addi R4 R0 172	[74]	[75, 75]	[]	[76]	[79]
Sd F12 0(R4)	[76]	[77, 77]	[81, 81]	[]	[81]
Sd F12 32(R4)	[78]	[79, 79]	[83, 83]	[]	[83]
Addi R5 R0 176	[78]	[79, 79]	[]	[80]	[83]
Sd F13 0(R5)	[80]	[81, 81]	[85, 85]	[]	[85]
Sd F13 32(R5)	[82]	[83, 83]	[87, 87]	[]	[87]
Addi R6 R0 180	[82]	[83, 83]	[]	[84]	[87]
Sd F14 0(R6)	[84]	[85, 85]	[89, 89]	[]	[89]
Sd F14 32(R6)	[86]	[87, 87]	[91, 91]	[]	[91]
Addi R7 R0 184	[86]	[87, 87]	[]	[88]	[91]
Sd F15 0(R7)	[88]	[89, 89]	[93, 93]	[]	[93]
Sd F15 32(R7)	[90]	[91, 91]	[95, 95]	[]	[95]
Addi R8 R0 188	[90]	[91, 91]	[]	[92]	[95]
Sd F16 0(R8)	[92]	[93, 93]	[97, 97]	[]	[97]
Sd F16 32(R8)	[94]	[95, 95]	[99, 99]	[]	[99]
Addi R15 R15 -1	[94]	[95, 95]	[]	[96]	[99]
Bne R15 R0 -8	[94]	[97, 97]	[]	[]	[99]

```
Addi R15 R15 -1              [98]          [99, 99]       []            [100]         [101]
Bne R15 R0 -8                [98]          [101, 101]     []            []            [102]
Addi R30 R0 123              [102]         [103, 103]     []            [104]         [105]

REGISTER VALUES
                  R0         R1         R2         R3         R4         R5         R6         R7
value:             0        160        164        168        172        176        180        184
                  R8         R9        R10        R11        R12        R13        R14        R15
value:           188          0         11          7         13         12         13          0
                 R16        R17        R18        R19        R20        R21        R22        R23
value:             7         13         13          7         11         14          8         13
                 R24        R25        R26        R27        R28        R29        R30        R31
value:             0          0          0          0          0          0        123          0
                  F0         F1         F2         F3         F4         F5         F6         F7
value:          0.0        2.0        3.0        4.0        5.0        6.0        7.0        8.0
                  F8         F9        F10        F11        F12        F13        F14        F15
value:          6.0        5.0        1.0       13.0        6.0        7.0       11.0        4.0
                 F16        F17        F18        F19        F20        F21        F22        F23
value:         10.0       30.0      -20.0       50.0       90.0      120.0      150.0       90.0
                 F24        F25        F26        F27        F28        F29        F30        F31
value:         10.0       20.0       30.0       40.0        0.0        0.0        0.0        0.0

NON-ZERO MEMORY VALUES
Address     Value
0           10.0
4           20.0
8           30.0
12          40.0
64          5.0
68          1.0
72          13.0
76          6.0
80          7.0
84          11.0
88          4.0
92          10.0
128         90.0
132         -20.0
136         50.0
160         5.0
164         1.0
168         13.0
172         6.0
176         7.0
180         11.0
184         4.0
188         10.0
192         5.0
196         1.0
200         13.0
204         6.0
208         7.0
212         11.0
216         4.0
220         10.0
```